

Aufgabe 1: RISC-V Konventionen

1. Mögliche Konventionen

Registersicherung:

1. Register sind unterteilt in: caller-saved vs. callee-saved (wer sichert und restauriert)
2. Gewisse Register dürfen / dürfen nicht vom Unterprogramm verändert werden (saved vs temporary)

Alignment:

(s. 08.93) Daten/Instruktionen dürfen nur auf Adressen abgelegt werden die

1. ein Vielfaches einer Zweierpotenz sind
2. ein Vielfaches der Größe des zugegriffenen Datums sind (natural alignment)

Stack:

1. Stapel wächst von oben nach unten (Stapelzeiger zeigt auf die niedrigste belegt Adresse) oder umgekehrt
2. Parameter liegen auf Stack vor oder nach Rücksprungadresse
3. Stapelzeiger zeigt auf letztes Element oder erste freie Speicheradresse

Parameterübergabe:

1. call-by-value (ein Wert wird übergeben)
2. call-by-reference (eine Referenz wird übergeben. In der Praxis: eine Adresse)
3. call-by-name (ein Ausdruck wird bei Aufruf des Unterprogramms neu ausgewertet)

2. RISC-V Konventionen

Registersicherung:

Register werden grob unterschieden:

1. saved registers `s0` - `s11`: Aufrufer darf vertrauen, dass Unterprogramm diese nicht verändert hat.
2. temporary registers `t0` - `t6`: Unterprogramm darf beliebig überschreiben.
3. Details hängen von genauer Architektur ab (32-Bit, 64-bit, I oder G Extension). Siehe Table 18.2 in RISC-V Calling Convention.

Register	ABI Name	Beschreibung	Saver
x0	zero	Hard-wired Zero	–
x1	ra	Return Address	Caller
x2	sp	Stack Pointer	Callee
x3	gp	Global Pointer	–
x4	tp	Thread Pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved Register / Frame Pointer	Callee
x9	s1	Saved Register	Callee
x10-x11	a0-a1	Function Arguments / Return Values	Caller
x12-x17	a2-a7	Function Arguments	Caller
x18-x27	s2-s11	Saved Registers	Callee
x28-x31	t3-t6	Temporaries	Caller

Table 1: RISC-V Calling Convention (nach Waterman et al., 2019)

Alignment:

1. Instruction Alignment: Vielfache von 4
2. Data Alignment: Keine Vorgabe, aber Konsequenzen, falls nicht naturally aligned (siehe vorherige Hausaufgabe, A-02c). Konsequenz: natural alignment wird empfohlen.

Stack:

1. Stapel von oben nach unten (Stapelzeiger zeigt auf die niedrigste belegte Adresse)
2. Stack pointer ist auf 16 Bytes aligned
3. Rücksprungadresse auf Stack zu legen ist Verantwortung des non-leaf Unterprogramms
4. Bei Spill in Parameterübergabe zeigt der stack pointer auf das erste auf dem Stack liegende Argument

Parameterübergabe:

Hauptquelle ist RISC-V Calling Convention. *ibid.*: *pointer-word*: register-width, also 32 Bits bei RV32.

1. Parameterübergabe wenn möglich in Registern und by-value.
2. Bei zu vielen Argumenten: Übergabe über Stack.
3. Integer-Register a0-a7 und floating-point Register fa0-fa7
4. Genaue Übergabe hängt wieder von spezifischer Architektur ab
5. Argumente kleiner als ein pointer-word werden in den least significant bits des entsprechenden Registers bzw. in kleineren Adressen auf dem Stack übergeben (wegen little-endian)
6. Argumente größer als doppelte Größe von pointer-word werden by-reference übergeben
7. Ähnliche Regeln für Rückgabe. Werte liegen in a0 (und a1 für 64-Bit Ergebnisse bei RV32).